Hadoop Lab 2 - Cluster Setup & MapReduce Log Analysis

Introduction

This project involves setting up and configuring a Hadoop cluster consisting of a master node and a worker node. Additionally, we developed and executed MapReduce programs to analyze large log files as part of the Lab 2 assignment.

Hadoop Cluster Configuration

1. Instance Setup

We created two instances using the prebuilt image **E516-Hadoop-Image-V4**:

- **node-master (Master Node)**
- **node-worker1 (Worker Node)**

2. Configuration Steps

a. Updating the Hosts File

- Edited /etc/hosts on both instances to add the private IP addresses of all nodes.
- Verified /etc/hostname on each instance for correctness.

b. Configuring Firewall Rules

To allow communication between nodes, we executed necessary firewall rules.

c. Hadoop Worker Configuration

- Modified the hadoop-3.4.0/etc/hadoop/workers file to include the worker node.

d. Enabling Web-Based Monitoring

Executed the following command to allow access from a specific IP:

sudo ufw allow from [Your Laptop IP Address]

MapReduce Programs

1. Checking Hadoop Services

Ensured that all Hadoop services were running using:

jps

2. Implemented Programs

Part 1: Finding Top K IP Addresses Per Hour

- Analyzed stat1 and stat2 example programs in Python and Java.
- Implemented a custom MapReduce program to:
    - Process log data and extract timestamps.
    - Count occurrences of IPs per hour.
    - Aggregate and sort the top k IPs per hour.

Part 2: Parallel Search with Time Period Filtering

- Extended the program to accept a time range from the user.
- Filtered log records within the specified time range.
- Output the top k IPs within the selected timeframe.
- Utilized structured intermediate data for efficiency.

Running the Program

1. Run the following command for both master and worker nodes: - *ssh exouser@<instance_id>*
2. Start Hadoop Cluster - start-all.sh
3. Create Map and Reduce Files

   You need three script files:

    - stat1.py (Mapper 1)
    - stat2.py (Mapper 2)
    - reducer.py (Reducer)

4. Create an input directory in HDFS and upload the log file:
    - *hdfs dfs -mkdir -p /input*
    - *hdfs dfs -put sample.log /input/*

5. Run Part 1 (Top K IPs per Hour)
   *hadoop jar /home/exouser/hadoop-3.4.0/share/hadoop/tools/lib/hadoop-streaming-3.4.0.jar *
   *-D mapred.reduce.tasks=1 *
   *-input /input/sample.log *
   *-output /output_stat1 *
   *-mapper "python3 map_stage_stat1.py 5" *
   *-reducer "python3 reduce_stage_stat1_stat2.py 5" *
   *-file /home/exouser/map_stage_stat1.py *
   -file /home/exouser/reduce_stage_stat1_stat2.py

6. Run Part 2 (Parallel Search with Time Period Filtering)
   *hadoop jar /home/exouser/hadoop-3.4.0/share/hadoop/tools/lib/hadoop-streaming-3.4.0.jar *
   *-D mapred.reduce.tasks=1 *
   *-input /input/sample.log *
   *-output /output_stat2 *
   *-mapper "python3 map_stage_stat2.py 03-04" *
   *-reducer "python3 reduce_stage_stat1_stat2.py 03-04 5" *
   *-file /home/exouser/map_stage_stat2.py *
   -file /home/exouser/reduce_stage_stat1_stat2.py

7. View Output
   hdfs dfs -cat /output_part1/part-00000
   hdfs dfs -cat /output_part2/part-00000

Conclusion

- Successfully configured a functional Hadoop cluster.
- Developed an efficient MapReduce program for log analysis.
- Applied firewall rules for secure communication.
- Enabled web-based monitoring.
- Demonstrated distributed computing power with Hadoop.
- Shelved the instances before submission.